SEMAINE 1 -- MERCREDI

Manipulation des Objets JavaScript

Programme de Renforcement DWWM -- Classe CodeWave

Objectifs de la séance
À la fin de cette séance, vous serez capable de :
Créer et manipuler des objets littéraux
Accéder aux propriétés (notation point et crochet) et vérifier leur existence
Parcourir un objet avec Object.keys(), Object.values(), Object.entries()
Déstructurer des objets (renommage, défaut, rest, imbrication)
Travailler avec des tableaux d'objets et résoudre des problèmes de logique métier

PARTIE 1 -- Documentation & Exercices Débutants

Pour chaque notion : lisez la syntaxe, testez l'exemple dans la console, faites l'exercice.

1.1 Créer un objet et accéder aux propriétés

JavaScript

```javascript
// Objet littéral
const produit = { id: 1, nom: 'Stylo', prix: 1.20, dispo: true };

// Notation point (clé connue)
console.log(produit.nom);          // 'Stylo'

// Notation crochet (clé dynamique ou avec espaces)
const champ = 'prix';
console.log(produit[champ]);       // 1.20

// Propriété absente => undefined  (pas d'erreur)
console.log(produit.stock);        // undefined
const s = produit.stock ?? 0;      // valeur par défaut avec ??

// Vérifier la présence d'une clé
'nom' in produit                   // true
produit.hasOwnProperty('ref')      // false

// Ajouter / modifier après création
produit.stock = 50;
produit['categorie'] = 'ecriture';
```

Exercice 1 -- Manipulation d'objet	Débutant	8 min

Partez de cet objet :

```javascript
const fiche = { prenom:'Bob', nom:'Dupont', age:34, ville:'Lyon' };
```

1. Affichez le nom complet (prénom + nom) en concaténant les deux propriétés
2. Écrivez getProp(obj, cle) qui retourne la valeur ou 'Inconnu' si la propriété est absente -- testez avec 'ville' et 'salaire'
3. Écrivez renommerCle(obj, ancienne, nouvelle) qui retourne un NOUVEL objet avec la clé renommée (sans muter fiche)

1.2 Object.keys(), Object.values(), Object.entries()

```javascript
const scores = { Alice: 92, Bob: 78, Clara: 85 };

Object.keys(scores)     // ['Alice','Bob','Clara']
Object.values(scores)   // [92, 78, 85]
Object.entries(scores)  // [['Alice',92],['Bob',78],['Clara',85]]

// Parcourir avec entries
Object.entries(scores).forEach(([nom, score]) => {
    console.log(`${nom} : ${score}/100`);
});

// Filtrer et reconstruire
const bons = Object.fromEntries(
    Object.entries(scores).filter(([_, s]) => s >= 80)
);  // { Alice: 92, Clara: 85 }
```

Exercice 2 -- Parcourir un inventaire	Débutant	8 min

```javascript
const inventaire = {
    stylo:  { prix: 1.5, stock: 200 },
    cahier: { prix: 3.5, stock: 85  },
    regle:  { prix: 0.8, stock: 0   },
    compas: { prix: 8.5, stock: 12  },
};
```

1. Affichez tous les noms de produits avec Object.keys()
2. Calculez la valeur totale du stock : somme de (prix * stock) pour chaque article
3. Créez prixSeuls : un objet { nom: prix } en utilisant Object.fromEntries() + map()

1.3 Déstructuration d'objets

```javascript
const user = { id:42, nom:'Alice', role:'admin', ville:'Paris' };

// Basique
const { nom, role } = user;

// Renommer
const { nom: prenom, ville: localisation } = user;

// Valeur par défaut
const { age = 0 } = user;          // age = 0 (absent)

// Rest
const { id, ...reste } = user;     // reste = { nom, role, ville }

// Dans les paramètres de fonction
function afficher({ nom, role }) {
    console.log(`${nom} est ${role}`);
}
afficher(user);

// Imbriquée
const { client: { nom: nomClient } } = { client: { nom: 'Dupont' } };
```

Exercice 3 -- Déstructuration en pratique	Débutant	8 min

```javascript
const commande = {
    id: 'CMD-001',
    client: { nom: 'Dupont', email: 'dupont@mail.com' },
    total: 18.50,
    livree: false
};
```

1. Déstructurez id et total en une ligne
2. Déstructurez le nom du client en imbriquant la déstructuration
3. Déstructurez en renommant total => montant et livree => estLivree

4. Écrivez resumeCommande({ id, client, total }) qui affiche : 'Commande CMD-001 - Dupont - 18.50 EUR'

1.4 Tableaux d'objets -- Le cas d'usage central

La combinaison tableau + objet est la plus courante : listes de produits, utilisateurs, commandes. On y applique map, filter, reduce, sort comme hier -- mais sur des objets.

```javascript
const employes = [
    { id:1, nom:'Alice', dept:'Dev',    salaire:3500 },
    { id:2, nom:'Bob',   dept:'Design', salaire:3200 },
    { id:3, nom:'Clara', dept:'Dev',    salaire:3800 },
];

// Noms des devs
employes.filter(e => e.dept === 'Dev').map(e => e.nom); // ['Alice','Clara']

// Mettre à jour sans muter (spread)
const maj = employes.map(e =>
    e.id === 2 ? { ...e, salaire: 3400 } : e
);

// Salaire moyen
const moy = employes.reduce((s, e) => s + e.salaire, 0) / employes.length;
```

Exercice 4 -- CRUD sur un tableau d'objets	Débutant	12 min

```javascript
let catalogue = [
    { id:1, nom:'Stylo bille',  prix:1.20, stock:150 },
    { id:2, nom:'Cahier A4',    prix:3.50, stock:45  },
    { id:3, nom:'Surligneur',   prix:2.10, stock:80  },
    { id:4, nom:'Post-it',      prix:3.80, stock:60  },
    { id:5, nom:'Ciseaux',      prix:6.30, stock:20  },
];
```

1. ajouterProduit(catalogue, produit) : id auto (max+1), retourne un nouveau tableau
2. rechercherParNom(catalogue, terme) : retourne les produits contenant terme (insensible à la casse)
3. modifierPrix(catalogue, id, nouveauPrix) : met à jour le prix, retourne un nouveau tableau
4. supprimerProduit(catalogue, id) : retourne un nouveau tableau sans le produit ciblé

Bonne pratique -- Immutabilité

Aucune fonction ne modifie le tableau d'origine.
CREATE : [...catalogue, nouveauProduit]
UPDATE : catalogue.map(p => p.id === id ? { ...p, ...modifs } : p)
DELETE : catalogue.filter(p => p.id !== id)

PARTIE 2 -- Challenges Intermédiaires

Consigne
Ces challenges ne sont pas guidés.
On vous donne une situation et un résultat attendu : c'est à vous de trouver la logique.
Écrivez votre raisonnement en commentaires avant de coder.

Exercice I1 -- Fusionner des objets sans écrasement	Intermédiaire	20 min

Implémentez fusionProfonde(obj1, obj2) : si une clé existe dans les deux et que les deux valeurs sont des objets, ils sont fusionnés récursivement -- pas écrasés.

Code	Resultat attendu
`fusionProfonde(`	
`  { a:1, b:{ x:1, y:2 } },`	
`  { b:{ y:99, z:3 }, c:4 }`	
`)`	`=> { a:1, b:{ x:1, y:99, z:3 }, c:4 }`
`fusionProfonde({nom:'Alice'},{age:30})`	`=> { nom:'Alice', age:30 }`

Indice
Pour chaque clé de obj2 : si les deux valeurs sont des objets non-null, récursion. Sinon, obj2 l'emporte.

Exercice I2 -- Pivot de données de ventes	Intermédiaire	20 min

Implémentez pivoter(releves) qui transforme un tableau de relevés en totaux par vendeur, et classement(releves) qui les trie du meilleur au moins bon.

JavaScript

```
const releves = [
    { vendeur:'Alice', mois:'Jan', ventes:12000 },
    { vendeur:'Bob',   mois:'Jan', ventes: 8500 },
    { vendeur:'Alice', mois:'Fev', ventes:15000 },
    { vendeur:'Bob',   mois:'Fev', ventes:11000 },
    { vendeur:'Clara', mois:'Jan', ventes: 9200 },
    { vendeur:'Clara', mois:'Fev', ventes:13400 },
];
```

Code	Resultat attendu
`pivoter(releves)`	`=> { Alice:27000, Bob:19500, Clara:22600 }`
`classement(releves)`	`=> [{vendeur:'Alice',total:27000}, ...]`

Exercice I3 -- Diff d'objets | Intermédiaire | 20 min

Implémentez diff(avant, apres) qui compare deux objets plats et retourne ce qui a changé, été ajouté, ou supprimé.

Code	Resultat attendu
`diff(`	
`  { nom:'Alice', age:28, ville:'Paris' },`	
`  { nom:'Alice', age:29, pays:'France' }`	
`)`	`=> {`
	`  modifie:  { age: { avant:28, apres:29 } },`
	`  ajoute:  { pays: 'France' },`
	`  supprime: { ville: 'Paris' }`
	`  }`

Exercice I4 -- Sérialiseur de formulaire | Intermédiaire | 25 min

Implémentez serialiser(paires) qui transforme un tableau de paires [clé, valeur] en objet structuré, en gérant les clés avec point (sous-objet) et les clés avec [] (tableau).

JavaScript

```
const paires = [
    ['prenom',        'Alice'          ],
    ['adresse.rue',   '12 rue du Code' ],
    ['adresse.ville', 'Paris'          ],
    ['tags[]',        'js'             ],
```

```
    ['tags[]',        'web'           ],
];
```

Resultat attendu

{ prenom:'Alice', adresse:{ rue:'12 rue du Code', ville:'Paris' }, tags:['js','web'] }

PARTIE 3 -- Challenges Avancés

Consigne

Pas d'indice, pas de structure donnée. Concevez la solution de A à Z.

Commentez votre logique avant d'écrire le code.

Exercice A1 -- Mini base de données en mémoire	Avancé	30 min

Implémentez createDB() qui retourne un objet avec les méthodes : insert, findAll, findOne, update, remove, query. Chaque 'table' est créée automatiquement à la première insertion avec un id auto-incrémenté.

```javascript
const db = createDB();
db.insert('users', { nom:'Alice', role:'admin' });
db.insert('users', { nom:'Bob',   role:'user'  });
db.insert('users', { nom:'Clara', role:'user'  });

db.findAll('users');
// [{id:1,nom:'Alice',role:'admin'},{id:2,...},{id:3,...}]

db.update('users', 2, { role:'moderateur' });
db.findOne('users', u => u.nom === 'Bob');
// { id:2, nom:'Bob', role:'moderateur' }

db.remove('users', 3);
db.query('users', u => u.role !== 'admin');
// [ { id:2, nom:'Bob', role:'moderateur' } ]
```

Exercice A2 -- TP Catalogue -- Version complète	Avancé	40 min

Construisez un système de gestion de catalogue complet. Toutes les fonctions retournent de nouvelles structures, pas de mutation.

```javascript
const produits = [
    { id:1, nom:'Stylo bille bleu',  cat:'ecriture', prix:1.20, stock:150, note:4.2 },
    { id:2, nom:'Stylo bille rouge', cat:'ecriture', prix:1.20, stock:0, note:4.0 },
    { id:3, nom:'Cahier A4 200p',    cat:'papier',   prix:5.50, stock:45, note:4.8 },
    { id:4, nom:'Bloc-notes A5',     cat:'papier',   prix:2.80, stock:30, note:3.9 },
    { id:5, nom:'Surligneur jaune',  cat:'ecriture', prix:1.80, stock:0, note:4.5 },
    { id:6, nom:'Surligneur rose',   cat:'ecriture', prix:1.80, stock:20, note:4.3 },
    { id:7, nom:'Ciseaux bureau',    cat:'bureau',   prix:6.50, stock:8, note:3.7 },
    { id:8, nom:'Agrafeuse',         cat:'bureau',   prix:9.90, stock:5, note:4.1 },
    { id:9, nom:'Post-it jaunes',    cat:'papier',   prix:4.20, stock:60, note:4.6 },
    { id:10,nom:'Agenda 2025',       cat:'agenda',   prix:12.0, stock:15, note:4.7 },
];
```

Fonctions à implémenter

1. rechercherProduits(produits, { categorie, prixMin, prixMax, enStock, noteMin }) -- critères tous optionnels
2. statsParCategorie(produits) -- { nbProduits, prixMoyen, stockTotal, noteMoyenne } par catégorie
3. top3(produits) -- les 3 produits avec la meilleure note parmi ceux en stock
4. appliquerPromotion(produits, categorie, pourcentage) -- prix réduits, pas de mutation
5. BONUS : genererRapport(produits) -- { totalProduits, produitsEnStock, valeurStockTotal, produitLePlusCher, produitMieuxNote }

Récapitulatif -- Objets JavaScript

Opération	Syntaxe	Notes
Accéder (fixe)	`obj.cle`	Clé connue à l'avance
Accéder (dynamique)	`obj[variable]`	Clé dans une variable
Ajouter / Modifier	`obj.cle = valeur`	Créée si inexistante

Opération	Syntaxe	Notes
Vérifier présence	`'cle' in obj`	Ou hasOwnProperty()
Lister les clés	`Object.keys(obj)`	Retourne un tableau
Lister les valeurs	`Object.values(obj)`	Retourne un tableau
Lister les paires	`Object.entries(obj)`	Tableau de [clé, val]
Reconstruire	`Object.fromEntries(tab)`	Inverse de entries()
Copie superficielle	`{ ...obj }`	Sous-objets partagés
Copie profonde	`structuredClone(obj)`	Indépendant à tous niveaux
Déstructuration	`const { a, b } = obj`	Extrait dans variables
Renommer	`const { a: x } = obj`	x reçoit la valeur de a
Valeur par défaut	`const { a = 0 } = obj`	Si a est undefined

Ressources recommandées
MDN -- Objets JS : https://developer.mozilla.org/fr/docs/Web/JavaScript/Guide/Working_with_objects
MDN -- Déstructuration : https://developer.mozilla.org/fr/docs/Web/JavaScript/Reference/Operators/Destructuring_assignment
JavaScript.info -- Objets : https://fr.javascript.info/object

Bon courage ! -- Said Aabilla, Formateur DWWM